

IHK-Leitfaden – Teil 2: Sortier- & Suchalgorithmen (FIAE, AP2) – Druckfreundlich

Nur Deutsch · Java-ähnlicher Pseudocode · ohne dunkle Hintergründe

Prüfungstipp: Erkenne Formulierungen wie „benachbarte Elemente vertauschen“ (BubbleSort), „Element an richtiger Stelle einfügen“ (InsertionSort), „Elemente nacheinander prüfen“ (lineare Suche) und „sortiertes Array, halbiere den Bereich“ (binäre Suche).

1. BubbleSort

Vergleicht benachbarte Elemente und vertauscht sie bei falscher Reihenfolge. Mehrfach wiederholen.

Pseudocode

```
void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int tmp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = tmp;
            }
        }
    }
}
```

Tipp: Laufzeit $O(n^2)$. Stabil, sehr einfach. „-i“ in der inneren Schleife spart Vergleiche am Ende.

2. InsertionSort

Fügt jedes Element an der richtigen Position in den bereits sortierten linken Teil ein. Stabil.

Pseudocode

```
void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

Tipp: $O(n^2)$, aber bei fast sortierten Arrays oft schneller als BubbleSort.

3. Lineare Suche

Prüft die Elemente der Reihe nach. Funktioniert auf unsortierten Arrays.

Pseudocode

```
int linearSearch(int[] a, int x) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == x) return i;
    }
    return -1;
}
```

Tipp: Laufzeit $O(n)$. Sobald gefunden → return.

4. Binäre Suche

Funktioniert nur auf **sortierten** Arrays. Halbiert den Bereich in jedem Schritt.

Pseudocode (iterativ)

```
int binarySearch(int[] a, int x) {  
    int left = 0, right = a.length - 1;  
    while (left <= right) {  
        int mid = (left + right) / 2;  
        if (a[mid] == x) return mid;  
        else if (a[mid] < x) left = mid + 1;  
        else right = mid - 1;  
    }  
    return -1;  
}
```

Tipp: Laufzeit $O(\log n)$. Vorher sortieren (z. B. mit InsertionSort), sonst liefert die binäre Suche falsche Ergebnisse.

5. Vergleich

Verfahren	Idee	Voraussetzung	Laufzeit	Stabil
BubbleSort	Nachbarn vergleichen/tauschen	–	$O(n^2)$	ja
InsertionSort	An richtige Stelle einfügen	–	$O(n^2)$	ja
Lineare Suche	Alle prüfen	–	$O(n)$	–
Binäre Suche	Bereich halbieren	sortiert	$O(\log n)$	–

6. Mini-Übungen (IHK-Stil)

A) Sortieren

Gegeben: `int[] a = {5, 2, 4, 3}`. Sortiere mit InsertionSort (Handsimulation) und notiere nach jedem äußeren Schleifendurchlauf den Zustand.

B) Suchen

Gegeben: sortiertes `int[] a = {2, 4, 7, 9, 12, 15}`. Suche 9 mit binärer Suche.
Welche `mid`-Werte entstehen?